# Solution: From SNAT to a Prethreaded Concurrent Server

## Part 1. Sockets Interface

The sockets interface gives user programs a file-descriptor-like abstraction for network communication. After a TCP connection is established, the client and the server can use ordinary Unix I/O operations, such as `read` and `write`, on socket descriptors.

The main socket calls have different roles on the client and server sides:

- `connect(clientfd, server_addr, server_addrlen)` is used by the client. It asks the kernel to establish a TCP connection to the server socket address. If the call succeeds, `clientfd` becomes the client's endpoint of the connection.
- `bind(listenfd, server_addr, server_addrlen)` is used by the server. It associates a socket descriptor with a local socket address, usually an IP-address:port pair.
- `listen(listenfd, LISTENQ)` turns the bound socket into a listening socket. It tells the kernel that this descriptor will be used to receive connection requests.
- `accept(listenfd, client_addr, client_len)` is used by the server. It blocks until a connection request is available, completes the server side of that connection, and returns a new connected descriptor.

The distinction between `listenfd` and `connfd` is essential:

- `listenfd` is the listening descriptor. It represents the server's passive endpoint for accepting future connection requests. The server usually creates it once and keeps it open for the lifetime of the server.
- `connfd` is a connected descriptor. It represents one established client-server connection. The server uses it to read data from and write data to exactly one client. A concurrent server can have many `connfd` values at the same time.

For this problem, after SNAT, the server sees the client as:

```
202.120.40.82:233
```

and the server endpoint is:

```
202.120.40.85:15213
```

Thus the TCP connection is identified on the public Internet by the socket pair:

```
(202.120.40.82:233, 202.120.40.85:15213)
```

At the private client itself, the client thinks the connection is:

```
(192.168.1.10:10086, 202.120.40.85:15213)
```

These two views differ because the SNAT router rewrites the source socket address on packets leaving the private network.

# Part 2. SNAT and IP Forwarding

The NAT-table entry is:

```
192.168.1.10:10086  <->  202.120.40.82:233
```

For a packet sent from the client to the server:

| Location | Source socket address | Destination socket address |
|---|---|---|
| Before SNAT, inside the LAN | `192.168.1.10:10086` | `202.120.40.85:15213` |
| After SNAT, on the public Internet | `202.120.40.82:233` | `202.120.40.85:15213` |
| As seen by the server | `202.120.40.82:233` | `202.120.40.85:15213` |

For a packet returned by the server:

| Location | Source socket address | Destination socket address |
|---|---|---|
| Before reverse translation, on the public Internet | `202.120.40.85:15213` | `202.120.40.82:233` |
| After reverse translation, inside the LAN | `202.120.40.85:15213` | `192.168.1.10:10086` |

SNAT means source network address translation. On the outgoing path, the router replaces the private source address and port:

```
192.168.1.10:10086
```

with its public address and a chosen public port:

```
202.120.40.82:233
```

The router also records this mapping in a NAT table. When the server replies to `202.120.40.82:233`, the router looks up the entry and rewrites the destination back to `192.168.1.10:10086`.

The server cannot directly see `192.168.1.10:10086` because `192.168.x.x` is a private address range. It is meaningful inside the LAN but is not globally routable on the public Internet. The public Internet routes the packet to `202.120.40.82`, and the NAT router is responsible for forwarding it to the private host.

# Part 3. Bounded Descriptor Buffer

The prethreaded server uses a producer-consumer pattern:

- The master thread is the producer. It accepts connections and inserts connected descriptors into the shared buffer.
- Worker threads are consumers. Each worker removes a descriptor from the buffer, services the client, and closes the descriptor.

A CSAPP-style bounded buffer can be represented as:

```
typedef struct {
    int *buf;          /* Buffer array */
    int n;             /* Maximum number of slots */
    int front;         /* buf[(front + 1) % n] is first item */
    int rear;          /* buf[rear % n] is last item */
    sem_t mutex;       /* Protects front, rear, and buf */
    sem_t slots;       /* Counts available empty slots */
    sem_t items;       /* Counts available descriptors */
} sbuf_t;
```

The initialization should be:

```
void sbuf_init(sbuf_t *sp, int n) {
    sp->buf = Calloc(n, sizeof(int));
    sp->n = n;
    sp->front = sp->rear = 0;
    Sem_init(&sp->mutex, 0, 1);
    Sem_init(&sp->slots, 0, n);
    Sem_init(&sp->items, 0, 0);
}
```

The meanings of the semaphores are:

- `mutex` is a binary semaphore. It protects the shared buffer metadata and prevents multiple threads from modifying `front`, `rear`, or `buf` at the same time.
- `slots` is a counting semaphore. Its value is the number of empty slots. It is initially `n` because the buffer starts empty.
- `items` is a counting semaphore. Its value is the number of available connected descriptors. It is initially `0` because no client has been accepted yet.

Correct insertion:

```
void sbuf_insert(sbuf_t *sp, int item) {
    P(&sp->slots);                          /* Wait for an empty slot */
    P(&sp->mutex);                          /* Enter critical section */
    sp->buf[(++sp->rear) % sp->n] = item;
    V(&sp->mutex);                          /* Leave critical section */
    V(&sp->items);                          /* Announce a new item */
}
```

Correct removal:

```
int sbuf_remove(sbuf_t *sp) {
    int item;

    P(&sp->items);                          /* Wait for an available item */
    P(&sp->mutex);                          /* Enter critical section */
    item = sp->buf[(++sp->front) % sp->n];
    V(&sp->mutex);                          /* Leave critical section */
    V(&sp->slots);                          /* Announce a new empty slot */
    return item;
}
```

The order of the `P` operations is important. In `sbuf_insert`, the producer must wait on `slots` before acquiring `mutex`. If it acquired `mutex` first and then blocked because the buffer was full, no consumer could acquire `mutex` to remove an item and create a free slot. That would be a deadlock.

Similarly, in `sbuf_remove`, a worker must wait on `items` before acquiring `mutex`. If it acquired `mutex` first and then blocked because the buffer was empty, the master thread could not acquire `mutex` to insert a descriptor. That would also be a deadlock.

The critical section should only protect the buffer manipulation. The server must not hold `mutex` while calling `echo(connfd)`, because `echo` can block for a long time waiting for client input. Holding the buffer mutex during `echo` would serialize the workers and destroy most of the benefit of concurrency.

A typical prethreaded server has this structure:

```
#define NTHREADS  8
#define SBUFSIZE  16

sbuf_t sbuf;

void *worker(void *vargp) {
    Pthread_detach(pthread_self());

    while (1) {
        int connfd = sbuf_remove(&sbuf);
        echo(connfd);
        Close(connfd);
    }

    return NULL;
}

int main(int argc, char **argv) {
    int listenfd, connfd;
    pthread_t tid;

    listenfd = Open_listenfd(argv[1]);
    sbuf_init(&sbuf, SBUFSIZE);

    for (int i = 0; i < NTHREADS; i++)
        Pthread_create(&tid, NULL, worker, NULL);

    while (1) {
        connfd = Accept(listenfd, NULL, NULL);
        sbuf_insert(&sbuf, connfd);
    }
}
```

# Part 4. Prethreading Advantages

Compared with an iterative server, a prethreaded server improves concurrency.
An iterative echo server has the following shape:

```
while (1) {
    connfd = Accept(listenfd, ...);
    echo(connfd);
    Close(connfd);
}
```

If one client connects and then stops sending data, the server can block inside `echo(connfd)`. During that time, the server is not accepting and servicing other clients. The fundamental flaw is that one slow client can delay all later clients.

A prethreaded server separates accepting from servicing:

- The master thread keeps accepting connections and inserting descriptors into a buffer.
- Worker threads service clients concurrently.

Thus, one worker may block on a slow client while other workers continue to serve other clients.

Compared with a thread-per-connection server, a prethreaded server improves resource management. A thread-per-connection server creates a new thread for each accepted connection:

```
while (1) {
    connfd = Accept(listenfd, ...);
    Pthread_create(&tid, NULL, thread, ...);
}
```

This design is concurrent, but it has two common costs:

1. Thread creation and destruction are not free. Creating a thread for every short connection can add significant overhead.
2. The number of threads can grow without a clear bound. A burst of clients can create many threads, increasing memory use, scheduling overhead, and context switching.

Prethreading creates a fixed pool of workers once at startup. This amortizes thread-creation cost and bounds the number of active worker threads.

Prethreading also avoids a classic argument-sharing bug. The buggy version is:

```
int connfd;

while (1) {
    connfd = Accept(listenfd, ...);
    Pthread_create(&tid, NULL, worker, &connfd);
}
```

Here all workers receive the address of the same local variable `connfd` in the master thread. Before a worker dereferences the pointer, the master thread might overwrite `connfd` with a later accepted descriptor. As a result, two workers may use the same descriptor, or a connection may never be serviced correctly.

In the prethreaded design, the descriptor value is copied into the bounded buffer. Workers remove descriptor values from the buffer, so they do not race on the master thread's stack variable.

# Part 5. Performance Trade-offs

Choosing `N`, the number of worker threads, involves a trade-off:

- If `N` is too small, the server may underuse the machine. For example, if many workers block on network I/O, additional workers could allow the server to continue serving other clients.
- If `N` is too large, the server may spend too much time on context switching and synchronization. More threads also consume more stack memory and can increase cache pressure.

For CPU-bound service routines, choosing `N` close to the number of CPU cores is often reasonable. For I/O-bound service routines, a larger `N` may help because many threads may be sleeping in `read`, `write`, or disk I/O. However, beyond a point, more threads mostly add scheduling overhead.

Choosing `B`, the descriptor-buffer size, also involves a trade-off:

- A larger buffer absorbs bursty arrivals. The master thread can keep accepting connections even if all workers are temporarily busy.
- A very large buffer can increase queueing delay. Accepted clients may wait a long time before a worker removes their descriptor.
- A larger buffer uses more memory, although descriptor buffers are usually small compared with thread stacks and connection state.
- A very small buffer applies backpressure quickly. This can keep latency more predictable under overload, but it may also make the master thread block in `sbuf_insert` more often.

Synchronization overhead should also be considered. Every connection goes through `P(&slots)`, `P(&mutex)`, `V(&mutex)`, and `V(&items)` on insertion, and through a similar sequence on removal. This cost is usually much smaller than network I/O for a normal echo or web server, but it can matter for extremely short requests or very high connection rates.

A good answer should connect the choice of `N` and `B` to the workload:

- Many slow clients: more workers and a moderate buffer can help.
- CPU-heavy request handling: too many workers can hurt performance.
- Bursty traffic: a larger buffer can absorb bursts.
- Strict latency target: avoid an excessively large buffer that hides overload by making clients wait.

# Part 6. Fairness and Starvation

The FIFO buffer provides FIFO ordering among descriptors that have already been inserted into the buffer. If descriptor `A` is inserted before descriptor `B`, then `sbuf_remove` will remove `A` before `B`.

However, this does not mean the whole server is perfectly fair.

Ordinary POSIX semaphores are synchronization primitives, not fairness policies. They ensure that:

- a thread waits when the required resource count is not available;
- a `sem_post` can wake a waiting thread;
- mutual exclusion works when a binary semaphore is used as a mutex.

They do not generally guarantee strict FIFO wakeup order among blocked threads. For example, if several workers are waiting on `items`, POSIX does not require the oldest waiting worker to wake first. The scheduler may repeatedly choose one thread over another. Therefore, worker-level starvation is possible in theory, especially with weak scheduling assumptions.

Client-level starvation is also possible in broader server behavior. For example, if all workers are occupied by clients that keep their connections open and never finish, later accepted clients can wait in the descriptor buffer indefinitely. If the buffer becomes full, the master thread may block in `sbuf_insert`, and new connection requests may remain in the kernel's listen queue or eventually be refused.

Thus the bounded buffer gives FIFO service order for queued descriptors, but it does not guarantee:

- FIFO ordering among workers waiting on a semaphore;
- fairness among clients before `Accept`;
- fairness if some clients hold workers for unbounded time;
- freedom from starvation under arbitrary scheduling.

Ways to improve fairness include:

- Use a fair semaphore, ticket lock, or condition-variable queue that wakes waiters in FIFO order.
- Add connection timeouts so one slow client cannot occupy a worker forever.
- Limit the amount of work or number of requests served per connection.
- Use separate queues or scheduling policies for different classes of clients.
- Monitor queue length and reject or shed load explicitly when the system is overloaded.

The key point is that correct synchronization is not the same as fairness. `mutex`, `slots`, and `items` make the bounded buffer correct, but an additional scheduling policy is needed if the server must provide strict fairness or bounded waiting.